

Day 4 · Topic 5: Quantization (AWQ INT4/INT8) & KV Cache Concepts

Week 1 — Local Inferencing Foundations

Goal: Reduce a model's VRAM footprint by ~4x using AWQ INT4 quantization, compare perplexity before and after, measure VRAM savings empirically, and deeply understand how the KV cache works and why it is the single most important memory optimization in production LLM serving.

PART A — Quantization: AWQ INT4 & INT8

What is Quantization?

Quantization reduces the numerical precision of model weights and/or activations from high-precision floats (FP16/FP32) to lower-precision integers (INT8/INT4). This directly reduces the bytes-per-parameter, shrinking both VRAM usage and the amount of data that must be transferred from GPU memory to compute cores on every forward pass. Since token generation is **memory-bandwidth bound** — not compute bound — reducing the bytes read per token translates directly into higher tok/s.

The Two Quantization Targets

Target	What gets quantized	Impact	Techniques
Weights	Model weight matrices (W)	Reduces VRAM & bandwidth at rest	AWQ, GPTQ, bitsandbytes
Activations	Runtime tensors (x, attn)	Reduces compute during forward	SmoothQuant, INT8 activations
KV Cache	Cached Key/Value tensors	Reduces KV cache VRAM	FP8 KV cache (vLLM 0.3+)

Today's focus is weight quantization — specifically AWQ INT4, which is the production sweet spot. We will cover activation and KV quantization when we get to vLLM in Week 2.

Naive Quantization vs AWQ — The Core Difference

Naive round-to-nearest (RTN) maps every FP16 weight to the nearest INT4 value using a global or per-channel scale factor. This causes large errors on *outlier weights* — the small fraction (~0.1-1%) of weights with unusually large magnitudes that have disproportionate impact on model output.

AWQ (Activation-Aware Weight Quantization) solves this by identifying salient weights — those that correspond to large activation channels — and applying a per-group scaling factor before quantization. The key insight is: rather than keeping salient weights in FP16 (which breaks the uniform INT4 layout), AWQ *scales up* salient weight channels so they survive INT4 quantization with minimal error, then absorbs the inverse scale into the next layer. All weights remain INT4, but quality is preserved.

Aspect	Naive RTN	GPTQ	AWQ
--------	-----------	------	-----


```

AWQ_DIR = "./phi2-awq-int4"
tokenizer = AutoTokenizer.from_pretrained(AWQ_DIR)

# Load the quantized model – note: much less VRAM than FP16
model = AutoAWQForCausalLM.from_quantized(
    AWQ_DIR,
    fuse_layers=True,          # fuse attention + MLP layers for speed
    trust_remote_code=True,
    device_map="cuda",
)

prompt = "The key principles of reliable distributed systems are"
inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
streamer = TextStreamer(tokenizer, skip_prompt=True, skip_special_tokens=True)

print(f"VRAM loaded: {torch.cuda.memory_allocated()/1e9:.2f} GB")
print("=== Output ===")

t0 = time.perf_counter()
with torch.no_grad():
    out = model.generate(
        **inputs,
        max_new_tokens=100,
        streamer=streamer,
        do_sample=True,
        temperature=0.7,
        pad_token_id=tokenizer.eos_token_id,
    )
elapsed = time.perf_counter() - t0
new_tokens = out.shape[1] - inputs["input_ids"].shape[1]
print(f"\n{new_tokens} tokens in {elapsed:.2f}s = {new_tokens/elapsed:.1f} tok/s")

```

A.4 INT8 Quantization with bitsandbytes (LLM.int8())

INT8 via bitsandbytes uses a **mixed-precision decomposition**: it identifies outlier activation dimensions at runtime, keeps them in FP16, and quantizes the rest to INT8. This is slower than AWQ but requires no calibration dataset and works on any HuggingFace model with a single flag.

```

from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch, time

MODEL_ID = "microsoft/phi-2"

# INT8 config
bnb_int8_config = BitsAndBytesConfig(load_in_8bit=True)

# INT4 config (NF4 – NormalFloat4, better distribution than plain INT4)
bnb_int4_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NormalFloat4 – best quality
    bnb_4bit_use_double_quant=True,    # quantize the quantization constants too
    bnb_4bit_compute_dtype=torch.float16,
)

def load_and_measure(config, label):
    torch.cuda.reset_peak_memory_stats()

```

```

t0 = time.perf_counter()
mdl = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, quantization_config=config,
    device_map="auto", low_cpu_mem_usage=True,
)
load_s = time.perf_counter() - t0
vram = torch.cuda.max_memory_allocated() / 1e9
print(f"{label:<20} Load: {load_s:.1f}s VRAM: {vram:.2f} GB")
return mdl

tok = AutoTokenizer.from_pretrained(MODEL_ID)

print(f"'Config':<20} {'Load time':>10} {'VRAM':>10}")
print("-" * 50)

mdl_int8 = load_and_measure(bnb_int8_config, "bitsandbytes INT8")
del mdl_int8; torch.cuda.empty_cache()

mdl_int4 = load_and_measure(bnb_int4_config, "bitsandbytes NF4")
del mdl_int4; torch.cuda.empty_cache()

```

A.5 Perplexity Comparison — Before vs After Quantization

```

import torch, math
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from datasets import load_dataset

def compute_perplexity(model, tokenizer, n_samples=50, max_len=512, device="cuda"):
    dataset = load_dataset("wikitext", "wikitext-2-raw-v1", split="test")
    texts = [t for t in dataset["text"] if len(t.strip()) > 100][:n_samples]

    total_nll, total_tokens = 0.0, 0

    for text in texts:
        enc = tokenizer(text, return_tensors="pt", truncation=True,
                        max_length=max_len).to(device)
        with torch.no_grad():
            out = model(**enc, labels=enc["input_ids"])
            nll = out.loss.item()
            total_nll += nll * enc["input_ids"].shape[1]
            total_tokens += enc["input_ids"].shape[1]

    return math.exp(total_nll / total_tokens)

MODEL_ID = "microsoft/phi-2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
results = {}

# FP16 baseline
model_fp16 = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, torch_dtype=torch.float16, device_map="cuda")
results["FP16 (baseline)"] = compute_perplexity(model_fp16, tokenizer)
del model_fp16; torch.cuda.empty_cache()

# INT8
model_int8 = AutoModelForCausalLM.from_pretrained(

```

```

MODEL_ID, quantization_config=BitsAndBytesConfig(load_in_8bit=True),
device_map="cuda")
results["INT8 (bitsandbytes)"] = compute_perplexity(model_int8, tokenizer)
del model_int8; torch.cuda.empty_cache()

# AWQ INT4 (if quantized model exists)
try:
    from awq import AutoAWQForCausalLM
    model_awq = AutoAWQForCausalLM.from_quantized("./phi2-awq-int4", device_map="cuda")
    results["AWQ INT4"] = compute_perplexity(model_awq, tokenizer)
    del model_awq; torch.cuda.empty_cache()
except Exception as e:
    print(f"AWQ model not found: {e}")

baseline = results.get("FP16 (baseline)", 1)
print(f"\n{'Config':<25} {'PPL':>8} {'vs FP16':>10}")
print("-" * 45)
for name, ppl in results.items():
    delta = f"+{((ppl/baseline)-1)*100:.1f}%" if name != "FP16 (baseline)" else "baseline"
    print(f"{name:<25} {ppl:>8.2f} {delta:>10}")

```

PART B — KV Cache: Concepts & Mechanics

The Problem: Quadratic Attention Cost

In transformer attention, every token must attend to every previous token. Without any optimization, generating token N requires recomputing the Key and Value projections for all N-1 previous tokens from scratch on every step — an $O(N^2)$ cost that makes long-context generation catastrophically slow.

The Solution: KV Cache

The KV cache stores the Key and Value tensors computed for every previous token in GPU memory. When generating token N, the model only needs to compute K and V for the *new* token, then retrieve all previous K/V pairs from cache and concatenate them. This reduces per-token generation from $O(N)$ to $O(1)$ — a massive speedup for long sequences.

What Gets Cached and Where

```
# In every transformer layer, attention computes:
#   Q = x @ W_q      (query – NOT cached, recomputed each step)
#   K = x @ W_k      (key   – CACHED after first computation)
#   V = x @ W_v      (value – CACHED after first computation)
#   Attn = softmax(Q @ K.T / sqrt(d_k)) @ V

# Cache layout per layer per token:
#   K tensor shape: [batch, num_heads, seq_len, head_dim]
#   V tensor shape: [batch, num_heads, seq_len, head_dim]

# KV Cache VRAM formula (from Day 3, now with full derivation):
#
# kv_bytes = 2          (K + V)
#           * num_layers (one KV pair per layer)
#           * batch_size (independent sequences)
#           * num_heads  (multi-head attention)
#           * head_dim   (dimension per head = hidden_dim / num_heads)
#           * seq_len    (max context tokens to cache)
#           * bytes_per_elem (2 for FP16, 1 for INT8/FP8)

def kv_cache_vram(num_layers, num_heads, head_dim,
                  max_seq_len, batch_size=1, dtype_bytes=2):
    bytes_total = (2 * num_layers * batch_size *
                  num_heads * head_dim * max_seq_len * dtype_bytes)
    return bytes_total / 1e9 # GB

# LLaMA-3 8B: 32 layers, 8 KV heads (GQA), head_dim=128, context=8192
kv_gb = kv_cache_vram(32, 8, 128, max_seq_len=8192, batch_size=1)
print(f"LLaMA-3 8B  KV cache (8K ctx, bs=1) : {kv_gb:.2f} GB")

kv_gb_bs8 = kv_cache_vram(32, 8, 128, max_seq_len=8192, batch_size=8)
print(f"LLaMA-3 8B  KV cache (8K ctx, bs=8) : {kv_gb_bs8:.2f} GB")

kv_gb_128k = kv_cache_vram(32, 8, 128, max_seq_len=131072, batch_size=1)
print(f"LLaMA-3 8B  KV cache (128K ctx,bs=1): {kv_gb_128k:.2f} GB")
```

Two Phases: Prefill vs Decode

Phase	When	What happens	KV Cache role
Prefill	Processing the prompt	All prompt tokens processed in parallel. KV pairs computed for every prompt token and stored in cache.	Populated
Decode	Generating each token	One new token generated per step using all cached KV-pairs.	Only the new token's KV is added.

This is why **TTFT is dominated by prefill time** (proportional to prompt length) and why subsequent tokens arrive faster and more evenly (pure decode).

Observing the KV Cache Effect

[illegible]

Observing KV Cache Hit Rate with Ollama

```
import requests, json

def query_ollama(prompt, model="llama3"):
    resp = requests.post("http://localhost:11434/api/generate", json={
        "model": model, "prompt": prompt, "stream": False,
```



```

        "options": {"num_predict": 30, "temperature": 0}
    }).json()
    return {
        "prompt_eval_count"    : resp.get("prompt_eval_count", 0),
        "prompt_eval_duration": resp.get("prompt_eval_duration", 1),
        "eval_count"           : resp.get("eval_count", 0),
        "eval_duration"         : resp.get("eval_duration", 1),
    }

# Same system prompt sent twice – 2nd call should be faster (prompt cached)
SYSTEM = "[SYSTEM] You are a helpful assistant. Always reply concisely. "
SYSTEM_LONG = SYSTEM * 20 # inflate the shared prefix

prompts = [
    SYSTEM_LONG + "What is 2+2?",          # cold – populates cache
    SYSTEM_LONG + "What is the capital of France?", # warm – shared prefix cached
    SYSTEM_LONG + "What is 7*6?",          # warm again
]

print(f"{'Prompt':<45} {'Prefill tok/s':>14} {'Gen tok/s':>10}")
print("-" * 72)
for i, p in enumerate(prompts):
    m = query_ollama(p)
    prefill_tps = m["prompt_eval_count"] / (m["prompt_eval_duration"] / 1e9)
    gen_tps     = m["eval_count"] / (m["eval_duration"] / 1e9)
    label = f"Call {i+1} ({'cold' if i==0 else 'warm'})"
    print(f"{label:<45} {prefill_tps:>14.0f} {gen_tps:>10.1f}")
print("\nExpect: warm calls show much higher prefill tok/s (cache hits)")

```

KV Cache Growth Visualization (Mental Model)

```

# KV cache memory grows linearly with generated tokens.
# This table shows memory growth for LLaMA-3 8B (32 layers, 8 KV heads, d=128, FP16)

import textwrap

def kv_at_step(n_tokens, layers=32, kv_heads=8, head_dim=128, bytes_pp=2):
    return 2 * layers * kv_heads * head_dim * n_tokens * bytes_pp / 1e6 # MB

print("LLaMA-3 8B – KV Cache Growth Over Token Generation")
print(f"{'Tokens generated':>18} {'KV Cache (MB)':>16} {'KV Cache (GB)':>15}")
print("-" * 52)
for n in [0, 128, 512, 1024, 2048, 4096, 8192, 16384, 32768, 131072]:
    mb = kv_at_step(n)
    print(f"{n:>18,} {mb:>16.1f} {mb/1024:>15.3f}")

print("\nNote: at 131K context (128K tokens) KV cache alone = ~8.5 GB")
print("This is why long context serving requires careful VRAM budgeting.")

```

Testing & Metrics

Full Quantization Comparison Benchmark

```

# Benchmark script: compare VRAM, speed, and quality across precision levels
# Run on a machine with at least one NVIDIA GPU (8+ GB VRAM recommended)

```

```

import torch, time, math, json
from transformers import (AutoModelForCausalLM, AutoTokenizer,
                          BitsAndBytesConfig, TextIteratorStreamer)

import threading

MODEL_ID = "microsoft/phi-2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
PROMPT = "The three most important factors in distributed system design are"
N_TOKENS = 80

CONFIGS = {
    "FP16": {"torch_dtype": torch.float16},
    "INT8": {"quantization_config": BitsAndBytesConfig(load_in_8bit=True)},
    "NF4 INT4": {"quantization_config": BitsAndBytesConfig(
        load_in_4bit=True, bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.float16)},
}

results = []
for label, kwargs in CONFIGS.items():
    torch.cuda.reset_peak_memory_stats()
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID, device_map="auto", low_cpu_mem_usage=True, **kwargs)
    model.eval()

    inputs = tokenizer(PROMPT, return_tensors="pt").to("cuda")
    streamer = TextIteratorStreamer(tokenizer, skip_prompt=True,
                                    skip_special_tokens=True)

    t0 = time.perf_counter()
    thread = threading.Thread(target=model.generate, kwargs=dict(
        **inputs, max_new_tokens=N_TOKENS, streamer=streamer,
        do_sample=False, pad_token_id=tokenizer.eos_token_id))
    thread.start()
    ttft = None
    for tok in streamer:
        if ttft is None: ttft = time.perf_counter() - t0
    thread.join()
    elapsed = time.perf_counter() - t0
    vram_mb = torch.cuda.max_memory_allocated() / 1e6

    results.append({
        "config" : label,
        "vram_mb" : vram_mb,
        "ttft_s" : ttft,
        "tok_s" : N_TOKENS / elapsed,
    })
    del model; torch.cuda.empty_cache()

print(f"\n{'Config':<14} {'VRAM (MB)':>10} {'TTFT (s)':>10} {'tok/s':>8}")
print("-" * 46)
for r in results:
    print(f"{r['config']:<14} {r['vram_mb']:>10.0f} {r['ttft_s']:>10.3f} {r['tok_s']:>8.1f}")

```

Key Metrics to Track

Metric	Formula / Tool	Target
VRAM savings	$(\text{fp16_vram} - \text{quant_vram}) / \text{fp16_vram} \times 100$	>50% for INT4
PPL degradation	$(\text{quant_ppl} - \text{fp16_ppl}) / \text{fp16_ppl} \times 100$	< 5% for AWQ INT4
Speed improvement	$\text{quant_tps} / \text{fp16_tps}$	1.2-2x for INT4 GPU
KV cache speedup	$t_{\text{no_cache}} / t_{\text{cached}}$	2-10x for long prompts
Cache hit effect	$\text{prefill_tps warm} / \text{prefill_tps cold}$	>2x on repeated prefix

Shell Verification Commands

```
# Monitor GPU VRAM in real-time during quantization / inference
watch -n 0.5 nvidia-smi --query-gpu=memory.used,memory.free,utilization.gpu \
    --format=csv,noheader

# Profile peak VRAM of a Python script
python -c "
import torch
torch.cuda.reset_peak_memory_stats()
# ... your model code here ...
print(torch.cuda.max_memory_allocated() / 1e9, 'GB peak VRAM')
"

# Check autoawq installed correctly
python -c "import awq; print(awq.__version__)"

# Check bitsandbytes GPU support
python -c "import bitsandbytes as bnb; print(bnb.__version__); bnb.cuda_setup.main()"
```

Key Takeaways

- **AWQ INT4 is the production sweet spot** for GPU inference: ~4x VRAM reduction with perplexity degradation typically under 3% vs FP16.
- **AWQ beats GPTQ** at the same bit-width because it protects salient weights via per-group scaling before quantization — all weights remain INT4 but quality is preserved.
- **bitsandbytes NF4** is the quickest path to INT4 inference — no calibration needed, just one flag — but is slower than AWQ at runtime due to mixed-precision decomposition.
- **The KV cache** is non-negotiable for token generation performance. Without it, each decode step costs $O(N)$ recomputation; with it, $O(1)$.
- **KV cache VRAM grows linearly** with sequence length and batch size. At 128K context a single LLaMA-3 8B request consumes ~8.5 GB of KV cache alone.
- **Repeated system prompts** benefit enormously from prefix caching (covered in Week 2 with vLLM) — the prefill phase is skipped entirely on cache hits.

Next topic: **Day 5 — Serving Fundamentals**: wrap HuggingFace pipeline in a FastAPI endpoint, add Pydantic validation, implement prompt engineering patterns, and handle context windows and token limits

correctly.